

MEERKATX

Unified Defense for Every Agent

A new AI-native security validation product, inspired by modern AppSec and agent-security research.

Management & Technical Committee Review | Team ExtraMile

Secure AI Before It Ships.



Approve MeerkatX as a new AI release-assurance MVP

The product combines broad AppSec testing, AI-native attack validation and traceable evidence in one experience.

WHY NOW

AI agents are being created rapidly by a broader community of builders; functional validation often comes before adversarial testing.

WHAT EXISTS

AppSec testing, prompt-injection and data-leak validation, a target onboarding UI and a shared attack-history view.

INSPIRATION

The team drew design inspiration from Strix capabilities and from AgentDojo and InjecAgent research, then built MeerkatX as a distinct product.

APPROVAL ASK

Harden the evidence model, prove Block → Fix → Retest → Pass, and validate the MVP across representative enterprise agents.

Decision requested: approve a controlled pilot-hardening phase, not a production rollout.

The speed of AI development is creating a new security gap

AI can build applications faster. Security testing has not caught up.

THE RISK

AI-built applications can look complete and still contain security weaknesses across access, authentication, injection, APIs, business logic, cloud, secrets and agent behavior.

THE BEHAVIOR GAP

Most creators first validate whether the agent answers correctly, automates the intended task and delivers the business outcome.

Fewer teams test how the agent behaves when prompts, tools, memory or external content are deliberately manipulated.

THE QUESTIONS

Can a malicious prompt redirect behavior?

Can confidential information be exposed?

Can tool permissions or workflow controls be bypassed?

Can misleading outputs still appear trustworthy?

Trust gap: “The agent works as expected” does not automatically mean “The agent is safe to deploy.”

Major application-security risks MeerkatX is designed to test

The AppSec lens is integrated into MeerkatX as a product capability, informed by established penetration-testing patterns.

ACCESS CONTROL

- Access Control
- IDOR
- Privilege escalation
- Authentication bypass

INJECTION ATTACKS

- SQL Injection
- NoSQL Injection
- OS Command Injection
- SSTI

SERVER-SIDE

- SSRF
- XXE
- Insecure Deserialization
- Remote Code Execution (RCE)

CLIENT-SIDE

- XSS — Stored / Reflected / DOM
- Prototype Pollution
- CSRF
- DOM vulnerabilities

AUTH & SESSION

- JWT vulnerabilities
- Session-management weaknesses
- Session fixation
- Credential-related attacks

API SECURITY

- Broken authentication
- Authorization issues
- Mass assignment
- Rate-limit bypass
- API endpoint exposure

Coverage is organized into reusable security lenses, while findings share one evidence and outcome model.

From business workflows to AI-agent attack chains

MeerkatX connects technical findings to the action, data or business process affected.

BUSINESS LOGIC

- Race conditions
- Workflow manipulation
- Workflow bypass
- Payment/business-process manipulation

INFRASTRUCTURE & CLOUD

- Cloud misconfigurations
- Exposed services
- Risky IAM permissions
- Infrastructure exposure

SECRETS & DATA

- Hardcoded credentials/secrets
- Secret leakage
- Exposed sensitive information

AI / AGENTIC SECURITY

- Prompt injection
- Indirect prompt injection
- Tool/agent misuse
- Agent instruction manipulation
- Tool-call authorization weaknesses
- Agent attack chaining

The differentiator is not only breadth of attacks; it is traceable evidence linking attack, execution path, impact and outcome.

A functional agent can still create enterprise risk

Security weaknesses can affect data, decisions, control boundaries and release confidence.

CONFIDENTIALITY

Sensitive information reaches an unauthorized user, tool or workflow.

INTEGRITY

Manipulated or incorrect output is treated as trustworthy.

CONTROL

An agent or connected tool performs an action outside its intended boundary.

ASSURANCE

Teams cannot reproduce what happened or support release decisions with evidence.

Management need: shift from functional confidence to evidence-based release assurance.

MeerkatX is a newly built, unified AI security product

Its design is inspired by Strix capabilities and agent-security research, but MeerkatX has its own product experience, orchestration and evidence model.



INTEGRATED APPSEC

Broad application, API, cloud, secret and business-logic testing capabilities.

AI-NATIVE TESTING

Prompt-injection, indirect instruction, data-leak and agent/tool misuse scenarios.

ATTACK TRACEABILITY

Attack, path, response, impact, outcome and evidence retained in a common history.

UNIFIED EXPERIENCE

Authorized target onboarding, assessment execution and release-oriented results through one UI.

Value proposition: one target, integrated security lenses, one traceable evidence body and one release-oriented result.

A new product with integrated security capabilities

Each security lens executes through MeerkatX orchestration and produces a shared, normalized evidence record.

USER EXPERIENCE

Target onboarding | Run control | Findings | Attack history

MEERKATX ORCHESTRATION

Target validation | Test planning | Execution | Result normalization

INTEGRATED SECURITY CAPABILITIES

AppSec | Prompt Injection | Data Leakage | Agent / Tool Misuse

TRACEABILITY & EVIDENCE

Attack source | Steps | Response | Impact | Outcome | Evidence

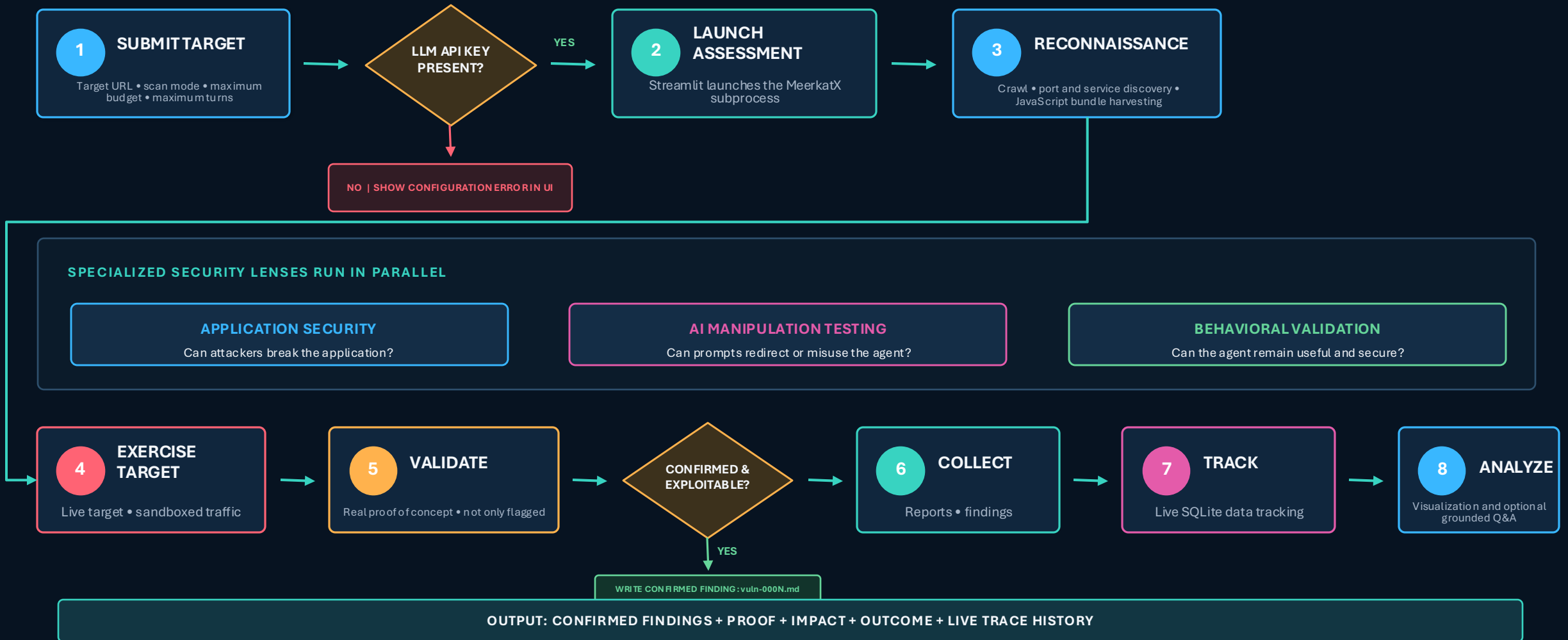
TARGET ENVIRONMENT

Web app | API | RAG assistant | Agentic app | DVAA demonstration

Design principle: capabilities may differ, but evidence, severity and release outcomes remain consistent.

From target submission to evidence-backed analysis

MeerkatX validates configuration, launches specialized testing, proves findings and preserves live assessment data.



Each specialized agent proves a different part of the risk story

Application security uncovers exploit paths; AI manipulation tests adversarial behavior; behavioral validation confirms utility and security outcomes.

APPLICATION SECURITY | CAN ATTACKERS BREAK THE APPLICATION?

ACCESS CONTROL

- Access Control
- IDOR
- Privilege escalation
- Authentication bypass

INJECTION ATTACKS

- SQL Injection
- NoSQL Injection
- OS Command Injection
- SSTI

SERVER-SIDE

- SSRF
- XXE
- Insecure Deserialization
- Remote Code Execution

CLIENT-SIDE

- Stored, Reflected and DOM XSS
- Prototype Pollution
- CSRF
- DOM vulnerabilities

AUTHENTICATION & SESSION

- JWT vulnerabilities
- Session-management weaknesses
- Session fixation
- Credential-related attacks

API SECURITY

- Broken authentication
- Authorization issues
- Mass assignment
- Rate-limit bypass
- API endpoint exposure

AI MANIPULATION TESTING

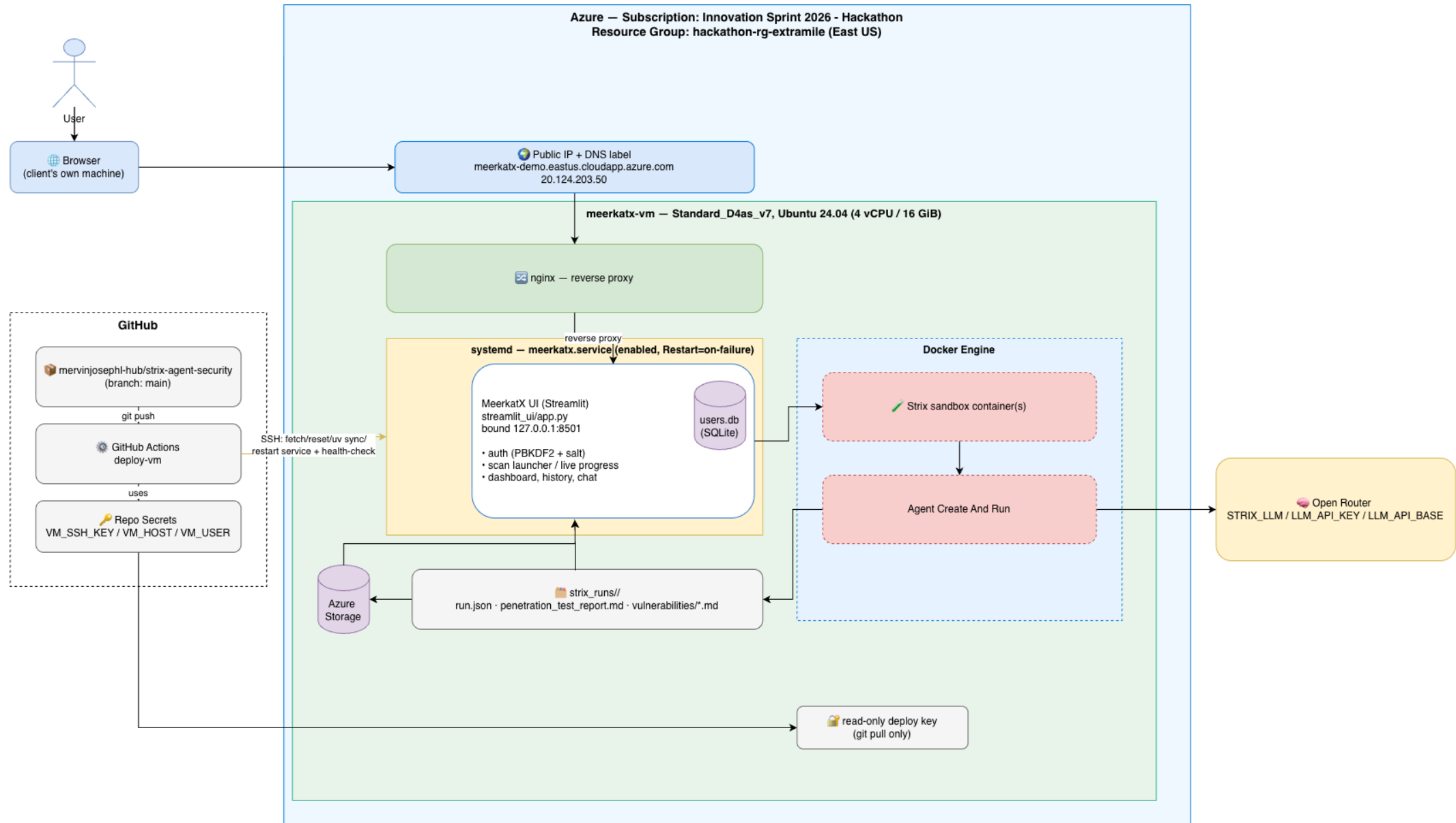
Indirect prompt injection • jailbreak and instruction override • unsafe tool invocation • data-theft actions

BEHAVIORAL VALIDATION

Stateful tool-use checks • business-logic and IDOR tests • attacker-goal detection • deterministic scoring

ONE RESULT MODEL: FINDING + PROOF + IMPACT + OUTCOME + TRACE HISTORY

MeerkatX — Architecture Diagram



Three sources shaped the MeerkatX product

Inspiration informs MeerkatX; it does not define MeerkatX as an extension of another product.

APPSEC / STRIX INSPIRATION

Autonomous security-testing patterns, broad vulnerability discovery and proof-oriented validation.

MeerkatX interpretation

Integrated AppSec capability within the MeerkatX workflow.

AGENTDOJO INSPIRATION

Stateful evaluation of tool-using agents under adversarial conditions.

MeerkatX interpretation

User-task, attacker-goal and state-impact checks.

INJECAGENT INSPIRATION

Indirect prompt injection and data-exfiltration test concepts.

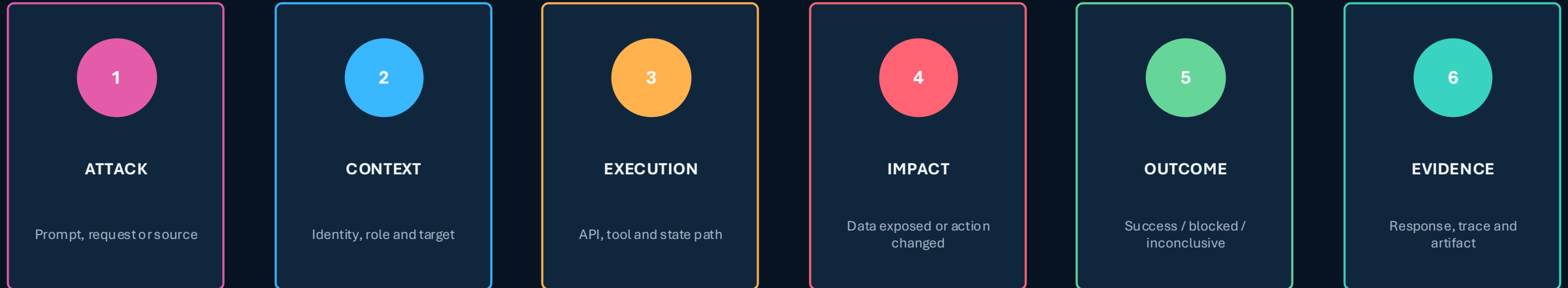
MeerkatX interpretation

Prompt, content and agent/tool misuse scenarios.

Claim discipline: MeerkatX is independently built and research-inspired; effectiveness must be demonstrated through repeatable evidence.

MeerkatX preserves the story of every attempted attack

A finding becomes actionable when teams can reproduce the path, understand the impact and verify the fix.



REPLAYABLE EVIDENCE

A confirmed failure becomes a regression test and is replayed after remediation.

Immediate proof target: Block → Fix → Retest → Pass using the same attack path.

What exists today — and what still requires approval

Clear boundaries increase credibility and keep the next phase focused.

WORKING MVP

- MeerkatX orchestration and unified UI
- Integrated AppSec capability
- Added prompt-injection and data-leak tests
- DVAA as controlled vulnerable target
- Attack history, impact and result capture
- Release-oriented result view

NOT YET PROVEN / MVP LIMIT

- Repeatability across multiple targets
- Enterprise identity and role controls
- CI/CD and ticketing integration
- Policy-as-code release gates
- Scale and performance
- Production hardening and operating model

Approval should fund validation and hardening of the evidence-to-release loop, not feature sprawl.

Prove the attack, the impact and the retest

DVAA is the controlled, known-vulnerable target used to demonstrate the MeerkatX workflow.

DVAA

**DAMN VULNERABLE
AI AGENT**

Intentionally vulnerable and isolated
for demonstration

1

BASELINE

Show the agent completing an expected task.

2

ATTACK

Run integrated AppSec and AI-native tests.

3

EVIDENCE

Open the trace and observed impact.

4

REMEDiate

Apply or simulate the corrected control.

5

RETEST

Replay the same path and show the changed outcome.

DEMO SUCCESS = SAME TARGET + SAME PATH + EVIDENCE OF BLOCK → FIX → RETEST → PASS

Measure trust outcomes — not the number of attacks

The next phase should have measurable committee-approved exit criteria.

REPLAYABLE EVIDENCE

% of confirmed findings with complete, reusable evidence

REGRESSION CONVERSION

% of confirmed findings converted into repeatable tests

RELEASE DECISION TIME

Time from assessment start to Pass / Remediate / Block

REMEDATION VERIFICATION

Time to replay the exact path and verify the fix

QUALITY OF DECISION

False-positive and false-block rates

AUTHORIZATION COVERAGE

Roles, tools, resources and high-impact actions represented

Exit criteria: repeatable evidence, regression tests, configurable policy and accountable ownership.

MeerkatX should not become another broad AI security suite

Focus on evidence, authorization and release assurance while integrating established controls.

MEERKATX OWNS

- Versioned evidence and replay
- Authorization-aware attack evidence
- Policy-based release decisions
- Regression test generation
- Fix verification
- Governance-ready evidence

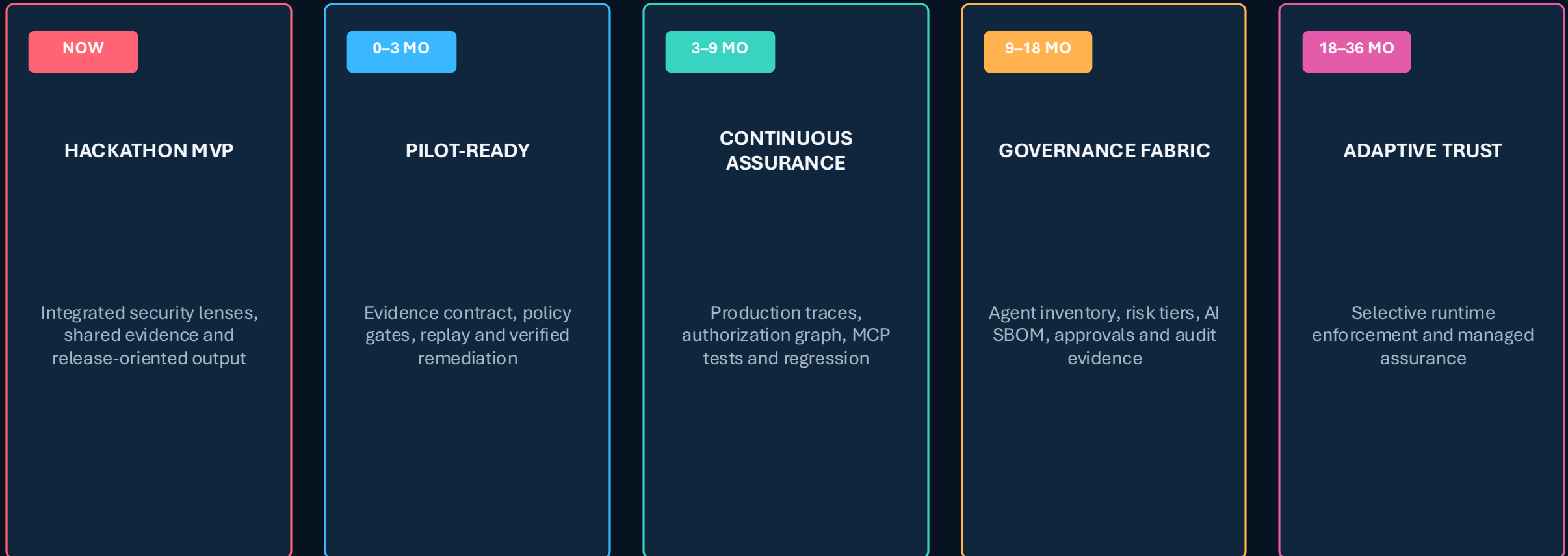
INTEGRATE / PARTNER

- Broad runtime firewalls and guardrails
- Full asset-discovery platforms
- SIEM, SOAR and GRC suites
- Enterprise DLP platforms
- General model-quality observability
- Mature enforcement ecosystems

North star: a vendor-neutral Agent Release Assurance and Continuous Trust layer.

From hackathon MVP to an enterprise AI trust layer

Planning horizons are recommendations, not committed delivery dates.



Grow MeerkatX around its evidence-and-authorization core, not around competitor feature replication.

Fund the next proof — not the final platform

The MVP demonstrates product direction; approval should focus on confidence, repeatability and enterprise fit.



HARDEN

Stabilize execution and version
the evidence contract.



PROVE

Demonstrate vulnerable and
corrected versions using the
same replay.



VALIDATE

Assess representative web, RAG
and tool-using agent targets.



OPERATIONALIZE

Define policy, privacy, ownership
and review controls for a pilot.

DECISION: APPROVE A CONTROLLED PILOT-HARDENING PHASE WITH DEFINED EXIT CRITERIA



MEERKATX

The Enterprise AI Trust Layer

Built as a new product, inspired by modern AppSec and agent-security research, and designed around integrated validation and replayable evidence.

Every confirmed issue can become a regression test. Every release can become a reusable evidence package.

Secure AI Before It Ships.

Team ExtraMile | MVP Approval Review